

## Práctico N° 5 – De Texto Caótico a Dataset Limpio

### 1. Contexto del Proyecto: "Sistema Profesional de Curación y Gestión de Datos"

Has sido contratado por una startup tecnológica que recibe datos de potenciales clientes de diversas fuentes (formularios web, chats de soporte, listas de eventos). Los datos llegan en un archivo de texto plano ([ingesta\\_cruda.txt](#)) con un formato caótico.

**Tu misión:** Desarrollar un programa en Python que automatice el proceso de **Ingesta, Limpieza, Validación, Gestión y Persistencia** de estos datos.

---

### 2. Requerimientos Técnicos (Integramos los vistos en U1 a U5)

#### PARTE 1: Ingesta Robusta

Implementar una función que lea el archivo fuente.

- **Manejo de Errores:** Debe soportar si el archivo no existe o si tiene errores de codificación (UTF-8).
- **Logging:** Cada vez que el programa inicie o encuentre un error, debe quedar registrado en un archivo `sistema.log`.

#### PARTE 2: El Motor de Limpieza - Regex

El archivo contiene líneas como: `Cli: Ana Lopez - Tel: 351-4567890 - Mail: ANA@GMAIL.COM`. Debes crear un módulo de limpieza que use **Expresiones Regulares** para:

1. **Extraer Nombres:** Limpiar prefijos y dejar el nombre en formato *Title Case*.
2. **Normalizar Teléfonos:** Extraer solo los números y validar que tengan exactamente 10 dígitos.
3. **Validar Emails:** Asegurar que tengan un formato @ válido y convertirlos a minúsculas.

#### PARTE 3: Gestión en Memoria

Los datos limpios deben almacenarse en una **Lista de Dicionarios**.

- **Menú Interactivo:** El programa debe tener un bucle principal (`while`) con las siguientes opciones:
  1. **Cargar y Limpiar:** Procesar el archivo de texto.

2. **Buscar:** Localizar un cliente por nombre (sin importar mayúsculas/minúsculas).
3. **Estadísticas:** Mostrar cuántos registros se cargaron, cuántos se descartaron y el porcentaje de emails de "gmail.com".
4. **Eliminar:** Quitar un cliente de la lista.
5. **Guardar y Salir.**

## PARTE 4: Calidad y Duplicados

- **Criterio de Unicidad:** No pueden existir dos clientes con el mismo email. Si el proceso de limpieza detecta un email duplicado, debe conservar sólo el primero y loguear el descarte.
- **Gestión de Faltantes:** Si a un registro le falta el teléfono o el email, el sistema debe decidir (según tu criterio documentado) si lo descarta o lo marca como "Desconocido".

## PARTE 5: Persistencia Estructurada

Al salir, el sistema debe exportar los datos limpios en dos formatos:

1. **CSV:** Para ser abierto en Excel (separador coma, encoding UTF-8).
2. **JSON:** Como respaldo de la estructura de datos completa.

---

## Forma de entrega

Para unificar y facilitar la corrección, todos los archivos solicitados deben ser comprimidos en un único archivo **.ZIP** nombrado de la siguiente manera: **Apellido\_Nombre\_TP5.zip**.

**El archivo comprimido debe contener obligatoriamente:**

1. **Código Fuente (.py):** Script principal modularizado (organizado en funciones), con nombres de variables descriptivos y comentarios que expliquen la lógica.
2. **Archivo de Log (sistema.log):** El log generado automáticamente por tu programa tras realizar una corrida de prueba completa con el archivo de ingesta.
3. **Informe de Reflexión Técnica (.pdf):** Un documento breve (máximo 1 carilla) con las respuestas a los siguientes puntos clave:
  - **Estructuras de datos:** ¿Por qué elegiste usar una lista de diccionarios en lugar de una lista de listas? Justifica en términos de legibilidad y acceso a los datos.
  - **Lógica Regex:** Explica paso a paso el patrón utilizado para validar el teléfono. ¿Cómo lograste filtrar los caracteres no numéricos?
  - **Gestión de Errores:** ¿Qué estrategia aplicaste ante datos faltantes o líneas corruptas? (¿Descarte, marcación o imputación?).

- **Escalabilidad:** ¿Cómo cambiaría tu código si el archivo tuviera 1 millón de filas? Menciona conceptos de la Unidad 5 como el uso de **Parquet** o la lectura por bloques (chunks).